① statement : line of code (instruction)

① normal (sequence) (print code → one by one)
② selection → if else if
③ loop i(iterative) statement (loop → iteration)

① What is looping statement?

① what is the existence of loop?

→ (block) → few line code
↓
1000 → iterate → 2y/m 3y/m

→ for this we are using the (for while)

1 to 10 →

print(1)
print(2)
print(3)
} more more to much confusing

1000
1000+1
1001

print(1)

100 ___ 10 →

0→

for i → in range (0, 100)
print(i) ⓪, ①, ②--⑩

index → 0
→ count

0 — ⑩

0 9 → ⑩

① statement :- line of code (instruction)

① normal (sequence) (print code → one by one)
② selection → if else if
③ loop :(iterative) statement (loop → iteration)

① what is looping statement?

① what is the existence of loop?

→ (block) → few line code
↓
1000 → iterate → [illegible]

→ for this we are using the (for while)

1 to 10 →

print(1)
print(2)
print(3)
.
} more more
to much confusing

1000
1000+1   print(1)   100 ___ 10 →
1001   0..
for i → in range (0, 100)
print(i) ⓪, ①, ②-- ⑩

index → 0
→ count   0 — ⑩
0 9 → ⑩

∞

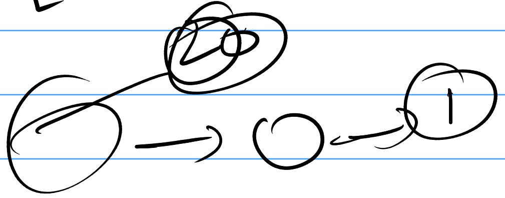
value = int('inf')

value = int("-inf")

second height

[1]

[10, 20, 30, 5]



→ comparison -∞, ∞

Two kind of loops:

① for loop
② while loop

① for loop:

→ sequence → str, list, tuple, ___)

→ we know How many iteration → for loop

→ [~~1000~~ 1, 2 - - - 10000) problem

why cond, condition
, iteration → ∞

99999 Sum

Two pointers

condition

for loop

②

[1 5000 10000]

9999

999

② wh con

prob

a = [10, 20, 30, 40, 50)
~~pmnt~~ for i in a:
print(i) →

for loop → conditon

conditon → while particuler true

↓

0 → tme exate

for loop
10
20
30
40

a => [10, 20, 30]

for i in a:
~~if~~ i == 30
print(i)
30

10

problem → even / odd → Pseudocode

a = [2, 1, 3, 4]

even → rem = 0
odd → rem = 1
→ remain

even = [ ]
odd = [ ]

even → 2%2=0
odd → 1%2=①

→ append →
[2]

Dry run

Bo hussh

↓

error find

a = [2, 3, 4, 4)
even = [ )
odd = [ )
for i n a:
if i%2==0:
even.append(i)

i → 2 [3]
2%2=0 ✓
even.append(2)
i → 3
if 3%2==0 ✗
else:
odd.append(3)

a → range } memory efficient)

range → for i in range (2, 1000)

print(i) 2, ... 999

range → sequence data type

2 1000

efficient — sta = 2
stop = 100
step = 1 } → number convert
~~po~~ item

[2, 60]

a = [1

a = [10, 20, 30]

~~Length~~ = len(a) mean ✗

3

while <3

→ iterator two → iterator → Advance python

↓

python for loop — iterator iterable

↓

list

↓

tuple

→ iterator → iter() } →
Next()

for i in a: a = [10, 20, 30, 40] → iterable

print(i) 10 20 30 40

iterator

→ iter(a)
↓
next 10
20
30
40

Stop →

iterator = iter(a)
Next = Next(iterator)
print(Next) 10
print(Next) 20

iter
↓
Next } to

while loop:

→ condition based loop

→ keep executing the code untill a condition True:

i <= 5

```
i = 2
while i<=5:
    print(i)
    i += 1
```

false

2
3
4
5

7

10+13

5+1

6

Dsa

Bran

brute

slid window sum

infinite loop

condition → always → True

```
i = 1
while i <= 5:
    print(i)
    i += 1
```

∞ — ∞

```
a = [10, 20, 30)
```

①

5+1

```
i = 0 1 2
n = len(a) = 3
while i<n:
    print(a[i])
    i += 1
```

operation

a[0)

10
20
30

b

break → loop terminate [illegible]

1, 2,

continue → iteration skip

```
for i in range(1,5)
    if i == 2:
        ~~continue~~
    print(i)
```

1, 5

i = 5 2

0

pass → nothing

```
def loop():
    pass
```

→

```
for i in rang(1,5):
    pass
    print
```

3

4

```
for i in range(1,5)
    print(i)
else:
    print("loop finished") loop
```

1-4

0 3

Nested loops:

for
for

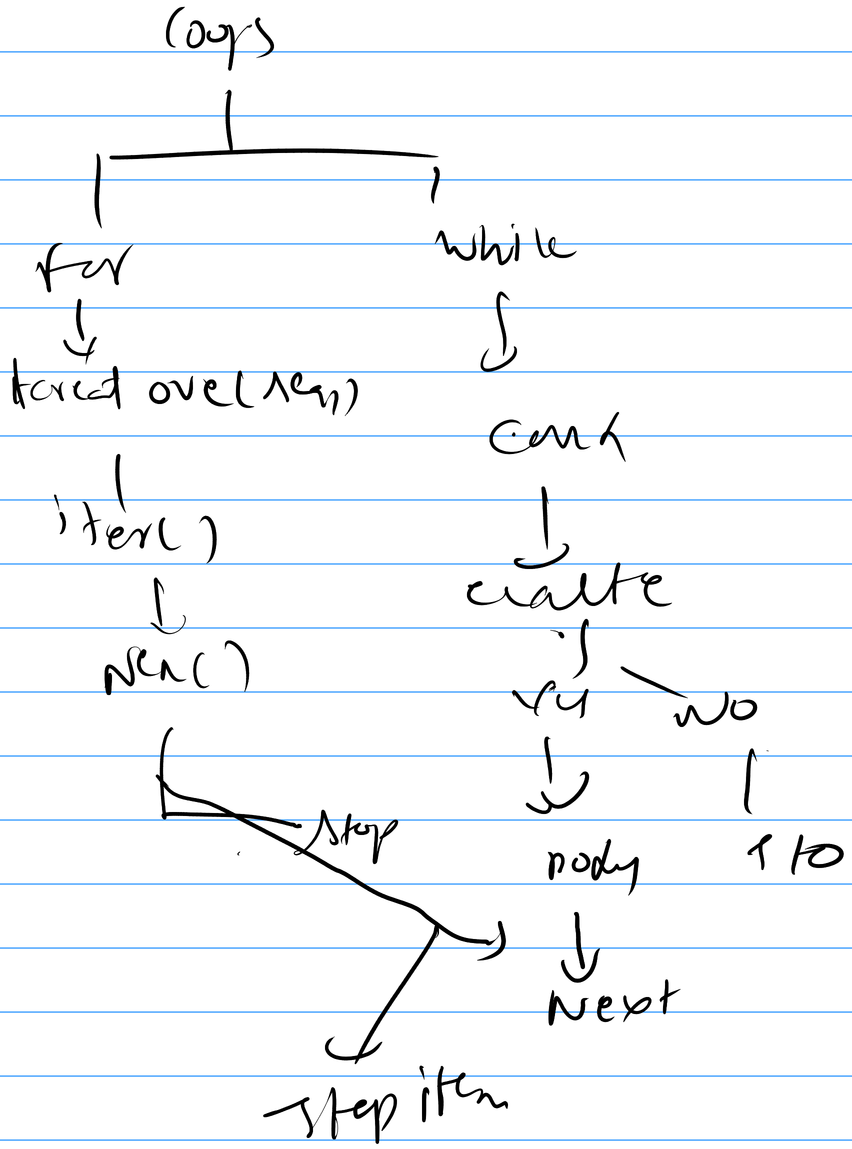
```
for i in range(3):
    for j in range(3):
        print(i,j)
```

0 1 2

n n n×2 9 3×3

```
0 0
0 1
0 2
1 0
1 1
1 2
2 0
2 1
2 2
```

```
0 0   1 0   2 0
0 1   1 1   2 1
0 2   1 2   2 2
```


loops
for
iterate over(seq)
iter()
next()
stop
step item
while
cond
evaluate
yes
no
body
Next